

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 40

Code of Conduct

I R.Jaideep(192525441)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: R.Jaideep

Assessment Tasks

Industry Problem Solving Task

1. An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.
2. A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.
3. A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.
4. A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.
5. A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application

Solution Design	25%	Solution is practical, scalable, and well reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

1. Big Data Analytics Pipeline for an E-Commerce Company

An e-commerce company generates huge amounts of data every day through **clickstream activities, online transactions, customer reviews, product searches, and user interactions**. Since this data is generated continuously from millions of users, a traditional database system may not be sufficient to store and analyze it efficiently. A **Big Data Analytics Pipeline** can be designed to collect, process, store, analyze, and visualize this information to support better business decisions.

Proposed Big Data Analytics Pipeline

The pipeline can consist of the following major stages:

1. Data Sources:

The first stage collects data from different sources such as website and mobile-app clickstreams, transaction systems, payment records, customer reviews, search history, social-media interactions, and product databases. Clickstream data may contain page views, clicks, session duration, and navigation paths, while transaction data contains orders, payments, and product details.

2. Data Ingestion:

A distributed data-ingestion layer continuously receives data from millions of users. Technologies such as **Apache Kafka** can be used for real-time streaming, while batch-based data can be imported from databases and files. The ingestion layer should be scalable so that it can handle sudden increases in user activity during festivals and special sales.

3. Data Storage:

The collected data can be stored in a distributed storage system such as a **data lake using cloud object storage**. Raw clickstream, transaction, and review data can be retained for future analysis. Structured and frequently accessed data can also be stored in distributed databases or data warehouses. Partitioning and replication can improve performance and reliability.

4. Data Processing and Cleaning:

The raw data is processed to remove duplicate records, handle missing values, identify invalid transactions, and standardize formats. Frameworks such as **Apache Spark** can perform large-scale batch and stream processing. Natural language processing can also be applied to customer reviews to identify positive, negative, or neutral opinions.

5. Analytics and Machine Learning:

After processing, analytics tools can be used to identify customer behavior and business trends.

Machine-learning models can support **product recommendations, customer segmentation, demand forecasting, fraud detection, sentiment analysis, and personalized marketing**. Real-time analytics can help detect important events immediately.

6. Visualization and Decision Making:

The final results can be presented through dashboards and reports using tools such as Power BI, Tableau, or cloud-based visualization services. Managers can monitor sales, customer behavior, product performance, inventory requirements, and customer satisfaction. The insights can then be used to improve pricing, marketing, inventory management, and customer experience.

Influence of the 5Vs on Architecture Design

The **5Vs of Big Data—Volume, Velocity, Variety, Veracity, and Value**—strongly influence the architecture of the proposed system.

1. Volume:

Millions of users generate terabytes or even petabytes of data. Therefore, the architecture must use **distributed and scalable storage** rather than relying on a single server. Cloud object storage, data lakes, and distributed databases allow storage capacity to increase as data grows.

2. Velocity:

Clickstream and transaction data can arrive continuously at very high speed. The architecture therefore requires **real-time data ingestion and stream processing**. Technologies such as Kafka and Spark Streaming can process incoming events with low latency, enabling applications such as real-time recommendations and fraud detection.

3. Variety:

The company receives structured data such as transaction records, semi-structured data such as JSON clickstream logs, and unstructured data such as customer reviews. The architecture should therefore support **multiple data formats and storage models**. A data lake is suitable because it can store different types of data before further processing.

4. Veracity:

Big data may contain incorrect, incomplete, duplicated, or misleading information. The pipeline must include **data validation, cleaning, deduplication, quality checks, and monitoring**. Reliable data is essential because poor-quality input can produce inaccurate analytics and business decisions.

2.Managing and Securing a Healthcare Data Ecosystem

A healthcare organization handles different types of sensitive information, including **structured patient records, semi-structured insurance claims, and unstructured medical images**. Since healthcare data contains confidential patient information, the organization needs an architecture that supports flexible storage, efficient processing, strong security, privacy, and regulatory compliance.

1. Data Collection and Classification

The first step is to collect data from different healthcare sources such as **Electronic Health Records (EHRs), hospital management systems, insurance platforms, laboratory systems, and medical imaging devices**. The data should be classified according to its type and sensitivity. Patient records can be classified as structured data, insurance claims as semi-structured data, and X-rays, CT scans, and MRI images as unstructured data.

2. Suitable Storage Architecture

Different storage technologies should be selected based on the nature of the data. Structured patient records can be stored in a **relational database** such as PostgreSQL or MySQL. Semi-structured insurance claims in formats such as JSON or XML can be stored in a **NoSQL database or document store**. Large medical images should be stored in secure **object storage or a healthcare data lake**. Metadata, such as patient ID, image type, date, and department, can be maintained separately to make images easier to search and manage.

3. Data Integration and Processing

A centralized data integration layer should connect the different healthcare systems. Standard healthcare formats such as **HL7 and FHIR** can be used to exchange patient information between applications. An ETL/ELT pipeline can clean, validate, transform, and integrate data before storing it. Data quality checks should identify duplicate records, missing information, and invalid values.

4. Security and Access Control

Security should be implemented at multiple levels. **Role-Based Access Control (RBAC)** should ensure that users can access only the information required for their responsibilities. For example, doctors may access clinical records, while insurance staff may access only relevant claims information. Strong authentication, preferably **multi-factor authentication (MFA)**, should be used for employees and administrators.

5. Encryption and Privacy

Healthcare data should be encrypted **both at rest and in transit**. Encryption protects databases, backups, and medical images if storage is compromised. Secure communication protocols such as TLS should protect information during transmission between hospitals, cloud systems, and authorized applications. Personally identifiable information should be protected using techniques such as **data masking, pseudonymization, or tokenization** when full patient identity is not required.

6. Backup and Disaster Recovery

The organization should maintain regular backups of patient records, claims, and medical images. Backups should be encrypted and stored separately from the primary environment. A disaster-recovery strategy should include **replication, defined Recovery Point Objectives (RPO), and Recovery Time Objectives (RTO)**. This allows healthcare services to continue even after hardware failures, cyberattacks, or natural disasters.

7. Monitoring and Auditing

Continuous monitoring should be implemented to identify unauthorized access, unusual data transfers, and suspicious user activity. Every important operation should be recorded in **audit logs**, including login attempts, patient-record access, data modification, and file downloads. Security alerts can help administrators respond quickly to potential breaches.

8. Compliance and Governance

The organization should establish policies for data retention, access, sharing, and deletion. Depending on the country and organization, applicable healthcare privacy and security regulations must be followed. Regular security assessments, vulnerability testing, employee training, and compliance audits should be conducted.

3. Big Data Churn Detection in Telecom

A telecom company generates enormous amounts of customer interaction data every day through **call records, SMS usage, mobile data consumption, recharge transactions, customer complaints, network usage, mobile applications, and customer-service interactions**. This data can provide valuable information about customer behavior and help the company predict which customers are likely to leave the service. However, analyzing such high-volume and continuously generated data requires a suitable Big Data architecture, analytics strategy, and integration approach.

1. Big Data Challenges

The first major challenge is **Volume**. Telecom companies may have millions of customers, with each customer generating thousands of interaction records. Call Detail Records (CDRs), billing information, browsing activity, and network logs can quickly reach terabytes or petabytes. Traditional databases may not efficiently handle this scale. Therefore, distributed storage such as a **cloud data lake, Hadoop Distributed File System (HDFS), or cloud object storage** should be used.

The second challenge is **Velocity**. Telecom data is generated continuously and must often be analyzed in real time. For example, a customer who suddenly reduces mobile-data usage or makes repeated complaints may be showing early signs of churn. Technologies such as **Apache Kafka** can collect real-time events, while **Apache Spark Streaming** can process them with low latency.

The third challenge is **Variety**. Customer information exists in different formats. Billing and customer-profile information are structured, application logs are often semi-structured, and customer reviews, emails, and service conversations are unstructured. A flexible data lake and NoSQL databases can store and manage these different formats before analytics is performed.

Another important challenge is **Veracity**, which refers to the quality and reliability of data. Telecom datasets may contain duplicate customer records, missing values, incorrect information, inconsistent formats, or inaccurate usage measurements. Data-cleaning, validation, deduplication, and quality-checking processes must therefore be included in the pipeline.

Security and privacy are also major concerns because telecom data contains sensitive customer information. Encryption, authentication, role-based access control, data masking, and audit logging should be applied to protect customer information and prevent unauthorized access.

2. Recommended Data Integration Strategy

The company should create a **centralized Big Data platform** that integrates information from multiple sources, including CRM systems, billing databases, CDR systems, network monitoring systems, mobile applications, customer-support platforms, and online channels.

For real-time data, **Kafka** can act as a message broker and collect customer events continuously. For historical or batch data, ETL/ELT pipelines can extract information from existing databases and load it into the data lake. A common customer identifier should be used to connect records from different systems.

After ingestion, **Apache Spark** can be used to clean, transform, and combine the datasets. The processed information can be stored in analytical databases or cloud data warehouses for faster querying and reporting. This architecture allows the company to combine historical customer behavior with real-time interactions.

3. Analytics Strategy for Churn Prediction

The next stage is to apply **descriptive, predictive, and real-time analytics**. Descriptive analytics can identify historical patterns among customers who previously churned. For example, churned customers may show declining data usage, frequent complaints, reduced recharge frequency, poor network experience, or repeated plan changes.

Predictive analytics can use machine-learning algorithms such as **Logistic Regression, Decision Trees, Random Forest, Gradient Boosting, and XGBoost** to calculate the probability that a customer will churn. Important features can include monthly usage, recharge frequency, average call duration, number of complaints, dropped calls, data consumption, service-plan changes, and customer-support interactions.

The model can divide customers into **low-, medium-, and high-risk churn groups**. Real-time analytics can then detect sudden behavioral changes. For example, if a high-value customer suddenly stops using data and submits several complaints, the system can immediately identify the customer as a high churn risk.

4. Business Integration and Retention

The churn prediction system should be connected to the company's **CRM and customer-service applications**. When a customer is identified as high-risk, an automated alert can be sent to the retention team. The company can respond with personalized offers, discounts, improved plans, technical support, or loyalty benefits.

4.Distributed File Systems and Data Integration in Smart City Analytics

A smart city project generates huge amounts of data from different sources such as **traffic sensors, air-quality monitoring devices, surveillance cameras, smart parking systems, weather stations, and public transportation systems**. These sensors continuously produce data that must be collected, stored, processed, and analyzed to improve city operations. Since the data is large in volume and generated at high speed, traditional centralized storage systems may not be sufficient. **Distributed File Systems (DFS)** and **data integration tools** provide an effective solution for large-scale smart city data analysis.

1. Role of Distributed File Systems

A Distributed File System stores data across multiple computers or servers instead of depending on a single storage system. Technologies such as **Hadoop Distributed File System (HDFS)** and cloud-based distributed object storage can be used to store smart city data.

The first advantage is **scalability**. Smart city sensors continuously generate data, and the volume can increase as more sensors are installed. A distributed file system allows additional storage nodes to be added when required. This provides the capacity needed to store large amounts of traffic records, air-quality measurements, and surveillance information.

Another important advantage is **fault tolerance**. In a distributed environment, data can be replicated across multiple nodes. If one server fails, another copy of the data remains available. This is particularly important for smart city applications because critical information such as traffic conditions and emergency events should not be lost because of a hardware failure.

Distributed file systems also support **parallel processing**. Large datasets can be divided into smaller blocks and processed simultaneously across multiple machines. For example, traffic data from different areas of a city can be processed at the same time. This reduces processing time and allows city authorities to obtain results faster.

2. Data Storage for Different Sensor Streams

Different smart city systems generate different types of data. Traffic sensors may produce structured numerical information such as vehicle count, speed, and congestion level. Air-quality sensors may generate measurements such as **PM2.5, PM10, CO₂, temperature, and humidity**. Surveillance systems generate large unstructured video files.

A distributed data lake can store these different datasets in their original formats. Structured sensor data can later be stored in databases or analytical warehouses, while large video files can remain in distributed object

storage. Metadata such as sensor ID, location, timestamp, and data type can be maintained to make the information easier to search and analyze.

3. Role of Data Integration Tools

Data integration tools connect data from different sources and make it available for unified analysis. Smart city data may come from thousands of sensors, IoT gateways, databases, mobile applications, and surveillance systems. Without integration, each system may operate separately, making it difficult to identify relationships between datasets.

Apache Kafka can be used as a real-time data ingestion platform. It can collect continuous sensor events and transmit them to processing systems. For example, traffic sensors can send vehicle-count information to Kafka, while air-quality sensors simultaneously send pollution measurements.

Tools such as **Apache NiFi** can help collect, route, transform, and manage data flows from different sources. ETL/ELT tools can also clean and transform historical data before loading it into a data lake or warehouse.

4. Data Processing and Analytics

After data is collected and stored, distributed processing frameworks such as **Apache Spark** can analyze large datasets. Spark can process traffic, pollution, and other sensor data in parallel.

For example, traffic data can be analyzed to identify congestion patterns. Air-quality data can be combined with traffic information to determine whether heavy traffic is associated with increased pollution in specific areas. Surveillance data can be integrated with traffic information to support incident detection and emergency response.

Real-time processing can also generate immediate alerts. If traffic sensors detect an unusual reduction in vehicle speed and surveillance systems detect an incident, the system can notify traffic authorities. Similarly, if air-quality sensors detect pollution levels above a defined threshold, an alert can be generated for the responsible department.

5. Benefits to Smart City Management

The combination of distributed storage and data integration provides several benefits. It enables **scalable storage, fault tolerance, faster processing, real-time monitoring, and unified analysis**. City administrators can use dashboards to monitor traffic congestion, pollution levels, public safety conditions, and infrastructure performance.

5. Secure Big Data Solution for Financial Transaction Analytics

A financial analytics firm processes a very large number of transactions every second. This data may include **customer details, account information, payment records, transaction amounts, timestamps, and merchant information**. Since financial data is highly sensitive, the organization must provide strong privacy and security while also supporting high-speed analytics. A secure Big Data architecture should therefore combine real-time processing, scalable storage, encryption, access control, monitoring, and compliance mechanisms.

1. Proposed Big Data Architecture

The proposed solution can be designed using a **secure cloud-based or hybrid Big Data architecture**.

Transaction data can be collected from banking systems, payment gateways, ATMs, mobile applications, and online payment platforms.

For high-velocity data, **Apache Kafka** can be used as the data-ingestion layer. Kafka can receive thousands or millions of transaction events continuously and distribute them to downstream processing systems. This provides scalable and reliable real-time data ingestion.

The incoming data can be processed using **Apache Spark Streaming** or another distributed stream-processing platform. The processing layer can identify suspicious transactions, calculate transaction patterns, and generate real-time fraud alerts. Historical data can be processed in batches for long-term financial analysis.

2. Secure Data Storage

Processed and raw data can be stored in a **secure data lake or distributed storage system**. Frequently accessed structured information can be maintained in a data warehouse or distributed database, while large historical datasets can be stored in cloud object storage.

Data should be classified according to its sensitivity. Personally identifiable information (PII), account numbers, and payment information should receive stronger protection than non-sensitive analytical information. Data retention policies should also be applied so that information is not stored longer than necessary.

3. Encryption

Encryption is one of the most important security controls. Data should be encrypted both **at rest and in transit**. Encryption at rest protects databases, data lakes, backups, and storage systems if an unauthorized person gains access to the storage infrastructure. Encryption in transit protects information while it moves between transaction systems, Kafka, processing engines, databases, and applications.

Strong key-management systems should be used to generate, store, rotate, and control encryption keys. Encryption therefore reduces the risk of sensitive financial information being exposed.

4. Access Control and Authentication

The system should use **Role-Based Access Control (RBAC)** and the principle of least privilege. Employees should receive only the permissions required for their jobs. For example, an analyst may access anonymized transaction information, while a security administrator may access security logs.

Multi-factor authentication (MFA) should be required for sensitive accounts and administrative operations. Privileged users should receive additional restrictions because compromised administrator accounts can provide attackers with extensive access to financial data.

5. Data Privacy and Anonymization

Privacy controls should be applied before data is made available for analytics. Sensitive information such as account numbers and customer identifiers can be **masked, tokenized, or pseudonymized**. Analysts should use anonymized or pseudonymized information whenever the actual identity of a customer is not required.

Data-access policies should define who can view, modify, export, or delete information. This reduces unnecessary exposure of customer information while still allowing useful analytics.

6. Monitoring, Auditing, and Threat Detection

Continuous monitoring should be implemented to detect unauthorized access and unusual transaction activity. All important activities, including logins, data access, modifications, and administrative actions, should be recorded in **tamper-resistant audit logs**.

Real-time analytics can also detect suspicious transaction patterns. For example, multiple high-value transactions from unusual locations or devices can trigger alerts. Machine-learning models can be used to identify abnormal behavior and support fraud detection.

7. Backup, Recovery, and Compliance

The organization should maintain **encrypted backups and geographically separated copies** of critical data. Disaster-recovery mechanisms should define suitable Recovery Point Objectives (RPO) and Recovery Time Objectives (RTO). This ensures that financial operations can continue after hardware failures, cyberattacks, or other disasters.